

ĐẠI HỌC BÁCH KHOA HÀ NỘI



BÁO CÁO BÀI TẬP VỀ NHÀ 05

Sinh viên thực hiện:	Ngô Đức Anh
-----------------------------	-------------

Giảng viên hướng dẫn:	Thầy Lê Bá Vui
Bộ môn:	Lập trình mạng
Trường:	Công nghệ thông tin & truyền thông

HÀ NỘI, 5/2026

Bài tập 03.01: File Server

1.1 Yêu cầu

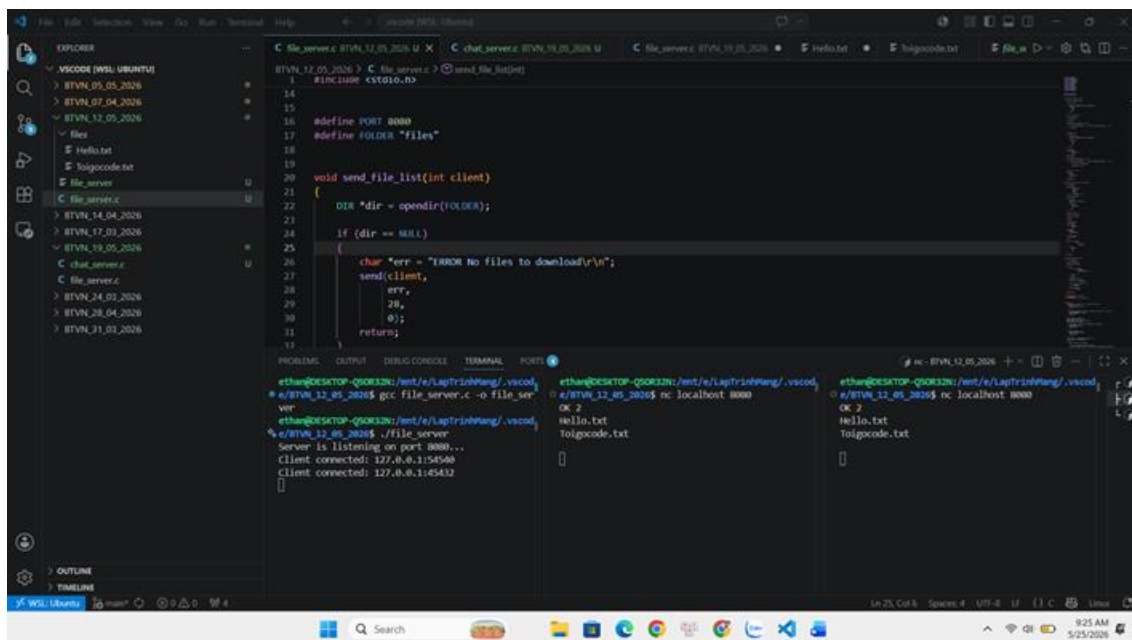
Sử dụng kỹ thuật đa tiến trình, lập trình ứng dụng file server đơn giản như sau:

- Khi client mới được chấp nhận, server sẽ gửi danh sách các file cho client. Các file này trong thư mục được thiết lập trên server. Dòng đầu danh sách là chuỗi “OK N\r\n” nếu có N file trong thư mục. Các tên file phân cách nhau bởi ký tự \r\n, kết thúc danh sách bởi ký tự \r\n\r\n.

- Nếu không có file nào trong thư mục thì server gửi chuỗi “ERROR No files to download\r\n” rồi đóng kết nối.

- Client gửi tên file để tải về, server kiểm tra nếu file tồn tại thì gửi chuỗi “OK N\r\n” trong đó N là kích thước file, sau đó là nội dung file cho client và đóng kết nối, nếu file không tồn tại thì gửi thông báo lỗi và yêu cầu client gửi lại tên file.

1.2.Kết quả thực nghiệm



```
1 #include <stdio.h>
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16 #define PORT 8080
17 #define FOLDER "files"
18
19
20 void send_file_list(int client)
21 {
22     DIR *dir = opendir(FOLDER);
23     if (dir == NULL)
24     {
25         char *err = "ERROR No files to download\r\n";
26         send(client,
27             err,
28             20,
29             0);
30         return;
31     }
32 }
```

```
ethan@DESKTOP-QG0K32N:/mnt/e/LapTrinhHuong/.vscode$ gcc file_server.c -o file_server
ethan@DESKTOP-QG0K32N:/mnt/e/LapTrinhHuong/.vscode$ ./file_server
Server is listening on port 8080...
Client connected: 127.0.0.1:5456
Client connected: 127.0.0.1:45432
ethan@DESKTOP-QG0K32N:/mnt/e/LapTrinhHuong/.vscode$ nc localhost 8080
OK 2
Hello.txt
Toigcode.txt
```

Ta minh họa mô hình client-server cho phép truy cập vào thư mục files và thao tác. Chạy chương trình file_server thì server hiện thị “Chat server listening on port 8080...”

Chạy ncat truy cập vào cổng 8080 thì client hiển thị 2 files đã có tồn tại và hiển thị “OK 2\r\n”.

The image shows a VS Code editor window with a C++ file server implementation. The code is in a file named `file_server.c` and is located in the `BTVN_12_05_2026` directory. The code defines a `send_file_list` function that sends a list of files to a client. The terminal output shows the server running and handling client requests. The client requests a file named `hello.txt` and the server responds with `OK 21` and the content of the file.

```
1  #include <stdio.h>
14
15
16 #define PORT 8080
17 #define FOLDER "files"
18
19 void send_file_list(int client)
20 {
21     DIR *dir = opendir(FOLDER);
22     if (dir == NULL)
23     {
24         char *err = "ERROR No files to download\n";
25         send(client,
26             err,
27             28,
28             0);
29         return;
30     }
31 }
```

Terminal Output:

```
ethan@DESKTOP-Q50K32N:/mnt/e/LapTrinhHuong/.vscode$ gcc file_server.c -o file_server
ethan@DESKTOP-Q50K32N:/mnt/e/LapTrinhHuong/.vscode$ ./file_server
Server is listening on port 8080...
Client connected: 127.0.0.1:55014
Client requests: hello
Client requests: hello.txt
[ ]
ethan@DESKTOP-Q50K32N:/mnt/e/LapTrinhHuong/.vscode$ nc localhost 8080
OK 2
hello.txt
Toigxcode.txt
[ ]
ethan@DESKTOP-Q50K32N:/mnt/e/LapTrinhHuong/.vscode$ nc localhost 8080
OK 2
hello.txt
Toigxcode.txt
[ ]
```

Ở client khi nhập sai tên file sẽ hiển thị văn bản báo lỗi. Ngược lại nếu nhập đúng sẽ hiển thị “OK 21\r\n” là số kí tự của file và nội dung của file.

Bài tập 03.02: Chat_group_server

2.1. Yêu cầu

Sử dụng kỹ thuật đa luồng, lập trình ứng dụng chat nhóm 2 người như sau:

- Mỗi client khi kết nối đến server sẽ được lưu vào hàng đợi, nếu số lượng client trong hàng đợi là 2 thì 2 client này sẽ được ghép cặp với nhau.
 - Khi 2 client đã được ghép cặp thì tin nhắn gửi từ client này sẽ được chuyển tiếp sang client kia và ngược lại.
 - Nếu 1 trong 2 client ngắt kết nối thì client còn lại cũng được ngắt kết nối
- ### 2.2. Kết quả thực nghiệm

The screenshot displays a C program in VS Code and its execution across five terminal windows. The program, located at `chat_room.c`, implements a multi-threaded chat server using `pthread`. It listens on `localhost 8080` and uses a queue to manage client connections. When two clients are connected, they are paired, and messages are relayed between them. The program includes a `chat_room(void *arg)` function that handles the pairing logic and message passing.

The five terminal windows show the following sequence of events:

- Terminal 1 (Server):
`pthread`
New client connected: 4
Waiting for another client to chat with 4...
- Terminal 2 (Client 4):
`nc -l -p 8080`
Partner found! You can start chatti ng.
- Terminal 3 (Client 5):
`nc localhost 8080`
Partner found! You can start chatti ng.
- Terminal 4 (Client 6):
`nc -l -p 8080`
Partner found! You can start chatti ng.
- Terminal 5 (Client 7):
`nc localhost 8080`
Partner found! You can start chatti ng.

The server terminal continues to show new client connections and waiting for partners, while the client terminals show the successful pairing of clients 4 and 5, 6 and 7, and the start of their chat session.

- 1) Ta minh họa trên mô hình client-server với 2 cặp client kết nối với nhau.
- 2) Hình ảnh trên minh họa cách các client bắt kết nối với nhau

The screenshot shows a VS Code editor with a C program for a chat server. The code is as follows:

```
29 void *chat_room(void *arg)
30 {
31     printf("Chat room created: %d <-> %d\n", c1, c2);
32     while (1)
33     {
34         FD_ZERO(&readfds);
35         FD_SET(c1, &readfds);
36         FD_SET(c2, &readfds);
37
38         int maxfd = (c1 > c2 ? c1 : c2) + 1;
39
40         int activity = select(maxfd, &readfds, NULL, NULL, NULL);
41
42         if (activity < 0)
43             break;
44
45         // client1 gửi
46         if (FD_ISSET(c1, &readfds))
47         {
48             int n = recv(c1, buffer, sizeof(buffer), 0);
49         }
50     }
51 }
```

The terminal windows show the following output:

```
ethan@DESKTOP-Q50K32N:/mnt/e/LapTr1:
nhang/.vscode/BTVM_19_05_2025 $ ./1
pthread
New client connected: 4
Waiting for another client to chat with 4...
New client connected: 5
Chat room created: 4 <-> 5
New client connected: 6
Waiting for another client to chat with 6...
New client connected: 7
Chat room created: 6 <-> 7

ethan@DESKTOP-Q50K32N:/mnt/e/LapTr1:
nhang/.vscode/BTVM_19_05_2025 $ nc localhost 8080
Partner found! You can start chatti
ng.
Hello Chao ban
Rat vui duoc gap ban
hello b

ethan@DESKTOP-Q50K32N:/mnt/e/LapTr1:
nhang/.vscode/BTVM_19_05_2025 $ nc localhost 8080
Partner found! You can start chatti
ng.
Hello Chao ban
Rat vui duoc gap ban
hello b

ethan@DESKTOP-Q50K32N:/mnt/e/LapTr1:
nhang/.vscode/BTVM_19_05_2025 $ nc localhost 8080
Partner found! You can start chatti
ng.
Hello Chao ban
Rat vui duoc gap ban
hello b

ethan@DESKTOP-Q50K32N:/mnt/e/LapTr1:
nhang/.vscode/BTVM_19_05_2025 $ nc localhost 8080
Partner found! You can start chatti
ng.
Hello Chao ban
Rat vui duoc gap ban
hello b
```

3) Khi thực hiện nhắn tin thì hai client cặp với nhau sẽ hiển thị nội dung trên khung chat.

The screenshot shows the same VS Code editor with the same C program. The terminal windows show the following output:

```
ethan@DESKTOP-Q50K32N:/mnt/e/LapTr1:
nhang/.vscode/BTVM_19_05_2025 $ ./1
pthread
New client connected: 4
Waiting for another client to chat with 4...
New client connected: 5
Chat room created: 4 <-> 5
New client connected: 6
Waiting for another client to chat with 6...
New client connected: 7
Chat room created: 6 <-> 7
Client 6 disconnected

ethan@DESKTOP-Q50K32N:/mnt/e/LapTr1:
nhang/.vscode/BTVM_19_05_2025 $ nc localhost 8080
Partner found! You can start chatti
ng.
Hello Chao ban
Rat vui duoc gap ban
hello b

ethan@DESKTOP-Q50K32N:/mnt/e/LapTr1:
nhang/.vscode/BTVM_19_05_2025 $ nc localhost 8080
Partner found! You can start chatti
ng.
Hello Chao ban
Rat vui duoc gap ban
hello b

ethan@DESKTOP-Q50K32N:/mnt/e/LapTr1:
nhang/.vscode/BTVM_19_05_2025 $ nc localhost 8080
Partner found! You can start chatti
ng.
Hello Chao ban
Rat vui duoc gap ban
hello b

ethan@DESKTOP-Q50K32N:/mnt/e/LapTr1:
nhang/.vscode/BTVM_19_05_2025 $ nc localhost 8080
Partner found! You can start chatti
ng.
Hello Chao ban
Rat vui duoc gap ban
hello b
```

4) Khi thực hiện ngắt kết nối 2 client thì không thể gửi tin nhắn và tự động ngắt kết nối ở client còn lại.

Bài tập 4.1: Chat Server

3.1. Yêu cầu

Sử dụng hàm `select()/poll()`, viết chương trình **chat_server** thực hiện các chức năng sau:

Nhận kết nối từ các client, và vào hỏi tên client cho đến khi client gửi đúng cú pháp:

“client_id: client_name”

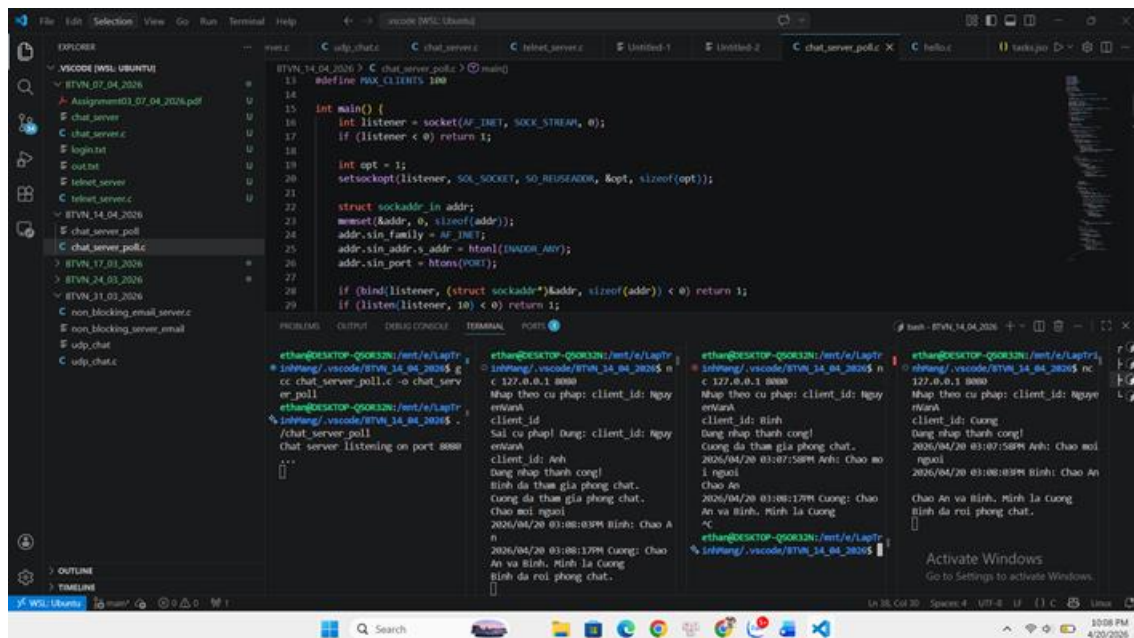
trong đó `client_name` là tên của client, xâu ký tự viết liền. Sau đó nhận dữ liệu từ một client và gửi dữ liệu đó đến các client còn lại, ví dụ: client có id “abc” gửi “xin chào” thì các client khác sẽ nhận được: “abc: xin chào” hoặc có thể thêm thời gian vào trước ví dụ: “2023/05/06 11:00:00PM abc: xin chào”.

Mô tả bài toán:

Bài toán yêu cầu xây dựng một chương trình **máy chủ trò chuyện (Chat Server)** sử dụng giao thức **TCP Socket** trên hệ điều hành Linux. Chương trình cho phép nhiều máy khách (client) kết nối đồng thời đến server và trao đổi tin nhắn trong cùng một phòng chat.

Server phải sử dụng cơ chế **I/O Multiplexing** thông qua hàm `poll()` để quản lý nhiều kết nối cùng lúc mà không cần tạo nhiều tiến trình hoặc nhiều luồng xử lý riêng biệt.

3.2. Kết quả thực nghiệm



Hình 1. Demo bài toán

- 1) Ta sẽ minh họa chương trình trên bằng cách kết nối 3 client gồm AN, BINH, Cuong và 1 server.
- 2) Chạy chương trình `chat_server` thì server hiện thị “Chat server listening on port 8080...”

- 3) Chạy netcat bằng cách thực hiện lệnh nc 127.0.0.1 8080, sau khi kết nối tới server thì hiển thị thông báo “Nhập theo cú pháp: client_id: NguyenVanA”.
- 4) Chương trình sẽ hiển thị thông báo lỗi nếu cú pháp đăng nhập không đúng. Nếu đăng nhập thành công thì bên client khác sẽ nhận được thông báo đã gia nhập phòng chat.
- 5) Nếu kết thúc chương trình ở client thì sẽ hiển thị thông báo đã rời khỏi phòng chat ở client khác.

Bài tập 4.2: Telnet Server

4.1. Yêu cầu

Sử dụng hàm select()/poll(), viết chương trình telnet_server thực hiện các chức năng sau:

Khi đã kết nối với 1 client nào đó, yêu cầu client gửi user và pass, so sánh với file cơ sở dữ liệu là một file text, mỗi dòng chứa một cặp user + pass ví dụ:

admin admin

guest nopass

...

Nếu so sánh sai, không tìm thấy tài khoản thì báo lỗi đăng nhập.

Nếu đúng thì đợi lệnh từ client, thực hiện lệnh và trả kết quả cho client. Dùng hàm system(“dir > out.txt”) để thực hiện lệnh.

dir là ví dụ lệnh dir mà client gửi. > out.txt để định hướng lại dữ liệu ra từ lệnh dir, khi đó kết quả lệnh dir sẽ được ghi vào file văn bản.

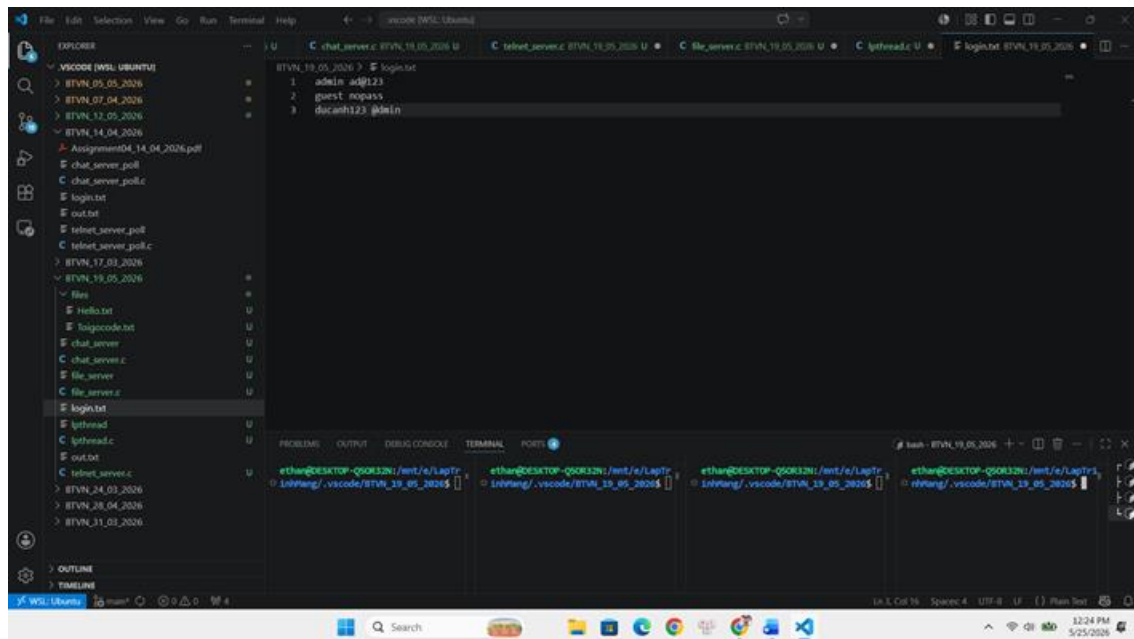
Mô tả bài toán

Bài toán yêu cầu xây dựng một chương trình **Telnet Server** hoạt động trên nền giao thức **TCP** bằng ngôn ngữ C trong môi trường Linux. Chương trình cho phép nhiều máy khách (client) kết nối đồng thời đến server thông qua mạng và đăng nhập bằng tài khoản được lưu trong file `login.txt`.

Server sử dụng hàm **pool()** để giám sát nhiều socket cùng lúc, nhờ đó có thể phục vụ nhiều client mà không cần tạo nhiều tiến trình hoặc nhiều luồng riêng.

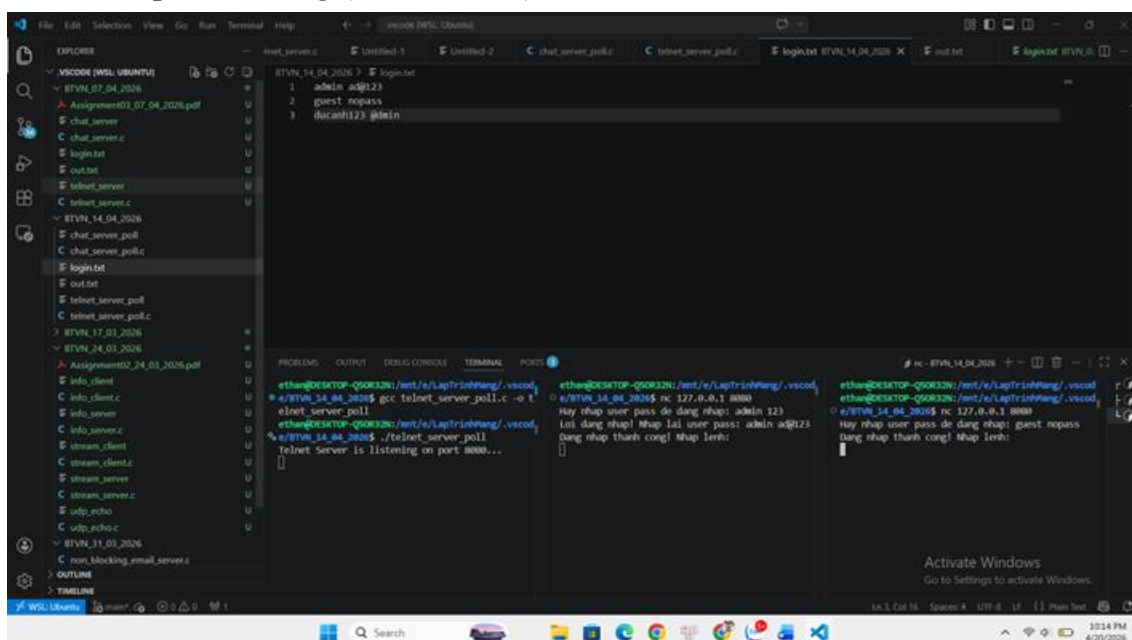
4.2. Kết quả thực nghiệm

- 1) Nội dung file login.txt mà chương trình sử dụng để tham chiếu khi client có thực hiện yêu cầu đăng nhập



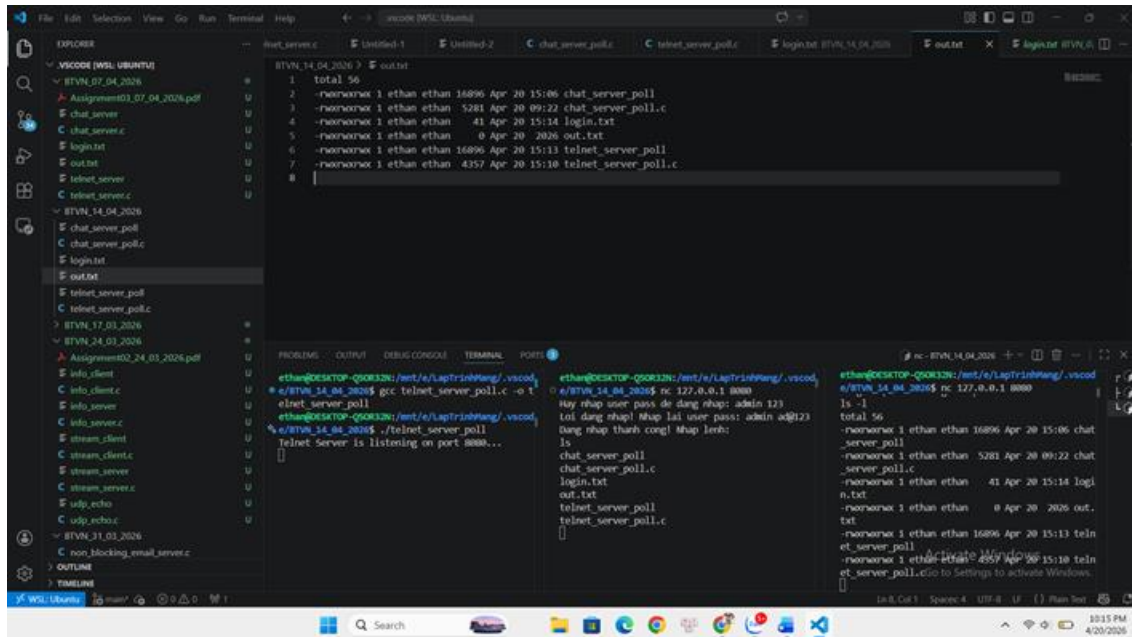
Hình 2. Nội dung file login.txt

- 2) Ta sẽ minh họa trên 3 terminal gồm 1 server và 2 client là admin và guest
- 3) Chạy chương trình telnet_server. Kết quả hiển thị “Chat server listening on port 8080...”
- 4) Chạy netcat bằng cách thực hiện lệnh nc 127.0.0.1 8080, sau khi kết nối tới server thì hiển thị thông báo “Hay nhập user pass de dang nhap”.
- 5) Trong trường hợp nhập sai thì client yêu cầu nhập lại. Nếu đúng thì hiển thị đăng nhập thành công (Xem hình 3.)



Hình 3. Demo đăng nhập

- 6) Khi nhập lệnh `ls -l` phía admin thì file `output.txt` sẽ hiển thị nội dung của lệnh thực hiện terminal



Hình 4. Demo lệnh `ls -l` và file `output.txt`

Bài tập 4.3: Time Server

Lập trình ứng dụng `time_server` thực hiện chức năng sau:

Chấp nhận nhiều kết nối từ các client sử dụng kỹ thuật multiprocessing.

Client gửi lệnh GET_TIME [format] để nhận thời gian từ server. format là định dạng thời gian server cần trả về client.

Các format cần hỗ trợ gồm:

dd/mm/yyyy– vd: 30/01/2023

dd/mm/yy– vd: 30/01/23

mm/dd/yyyy– vd: 01/30/2023

mm/dd/yy- vd: 01/30/23

Cần kiểm tra tính đúng đắn của lệnh client gửi lên server.



Bài tập 4.4: HTTP Server

The image shows a development environment with VS Code and a web browser. The VS Code editor is open to a file named `http_server.c` in the `BTVN_19_05_2026` project. The code is a C program that implements a simple HTTP server using `pthread` for threading. It listens on port 8080 and responds to GET requests with a "Hello everyone" message in HTML. The terminal output shows the server running and a log message indicating a client connection.

```
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

The terminal output shows the server running and a log message indicating a client connection:

```
ethan@DESKTOP-Q5OR32N:/mnt/e/LapTrinhWang/.vscode/BTVN_19_05_2026$ ./http_server  
server  
sec-ch-ua-mobile: ?0  
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8  
Sec-Fetch-Site: same-origin  
Sec-Fetch-Mode: no-cors  
Sec-Fetch-Dest: image  
Referer: http://localhost:8080/  
Accept-Encoding: gzip, deflate, br, zstd  
Accept-Language: en-US,en;q=0.9  
[Log] Thread 138513265710784 accepted client socket: 4
```

The web browser (Chrome) shows the response "Hello everyone" and the status "Processed by Thread ID: 138513282496192".